

① Learning python

i) Variables

- Variables can be defined as a container which is used to storing data in python.

• example: `a = 'saval'`

→ Here 'a' is the variable which stores saval.

* Legal Variables Names:

`fruitname = "apple"`

`fruit_name = "apple"`

`_fruitname = "apple"`

`fruitName = "apple"`

`FRUITNAME = "apple"`

`fruitname2 = "apple"`

* ILLEGAL Variables naming :-

`fruitname = "apple"` (blank space in starting)

`fruitname- = "apple"` ('-' cannot be used)

`fruit name = "apple"` (blank spaces)

`2fruitname = "apple"` (cannot start with number)

Storing Different types of Data

`number = 33` # integer

`number = 33.8` # float

`number = "33"` # string

`number = "a"` # char

* Assigning Many values to multiple variables:

→ python allows user to assign values to multiple variables in one line.

Syntax:

```
car1, car2, car3 = "toyota", "Honda", "ford"
```

* Checking the data type of variable stored value

→ The built-in function `type()` is used to check the datatype.

Syntax:

```
Name = "Nhanush"
```

```
print(type(Name))
```

output: 'str' (string)

④ Storing paragraphs in variables:

→ User can use triple double quotations mark to store paragraph.

For eg: `paragraph = """ ..... """`

④ ~~Can~~ Variables are case sensitive in py.

```
a = "Hero"
```

```
A = "Hero"
```

These "a" and "A" are two different variables.

2. Print Statement.

→ We use `Print()` function to display the output values.

Syntax:

`Print("saval learn python")` # print string

`Print(12)` # print integer value.

`Print(12.5)` # print float value

Output:

Saval learn python

12

12.5

③ Printing in new line:

`Print("Myname is \n Saval")`

Output:

Myname is

Saval

[\n starts a newline]

④ Common mistakes:

- Mistake 1 → Forgetting the comma betⁿ items

eg: `Print("Age:" 20)`

Always place a comma betⁿ separate items

eg: `Print("Age:", 20)`

- Mistake 2 → Joining text and a number with '+'

eg: `Print("Age" + 20)`

This is wrong because '+' cannot join text and a number

3) Comments

→ Comments are the notes which is totally ignored by the python interpreter during execution.

Syntax:

```
Print ("Hello world")
```

```
# Here helloworld is printed (This is comment)
```

* Single line and multiline comments

→ is used for single line comment.

Note: Python doesn't have multiline comments so programmers use consecutive # symbols or unassigned triple quotes (""")

* Comment can be used to describe a program

for example:

```
a = 5 [ # integer values are assigned ]  
b = 6
```

```
total = a + b # total value is calculated
```

```
print(total) # displays the total value
```


4) Data types:

→ used to define the nature of a variables.

Some of the data types are :- string, integer, float, ~~bo~~ boolean.

④ Examples:

a = 5 (this is a integer)

b = 5.5 (this is a float value)

c = "A" (this is a char)

d = "hello" (this is a string)

choice = True (this is a boolean value)

④ Checking datatype in python

→ User can use the builtin function `type()` which identifies the type of data used.

④ Behaviour of Operations According to Datatype:

→ If we add two strings, the strings ~~are~~ concatenates whereas when we add two integer or integer and float, it performs addition:

Example:

a = 5

b = 5

print(a+b)

x = "5"

y = "5"

print(x+y)

Output:

10

55

Important Notes:

- Every value has exactly one type - can check using `type()`.
- '+' means add for numbers but "join" for the text and "combine" for lists.
- None is different from 0, and False.
- True and False behave like 1 and 0.

5) User Input:

→ It is used to take an actual real information from the users, instead of working with fixed values.

Syntax:

```
name = input("enter your name")
```

This will prompt user to input their name.

```
age = int(input("enter your age"))
```

This will help to take input in integer as it is age.

Notes:

~~input()~~ input() always returns the string as default. We can convert to int, float according to our need.

Example:

```
value = input("Type a number to double:")
```

```
doubled = value * 2
```

```
print(doubled)
```

In this case,

→ If we entered 5, we will expect 10 as our output but the output will be 55 because it is taken as a string.

⊗ Correct way:

```
value = int(input("Type a number to double"))
```


6) Strings and String Concatenation

→ It is the Python's way to represent a text.

* Strings in python are surrounded by either single quotation or double quotation marks.
Example:

```
a = "saval" # This is a string.
```

or,

```
a = input("enter your name")
```

This prompt user to input string.

* String length:

→ len() function is used to calculate the length of the string.

Syntax:

```
name = "saval"
```

```
print(len(name))
```

Output = 5

* Character Accessing and Indexing In String

→ User's can access individual character in a string by using [] containing an Integer Index.

Example: name = "saval"

```
Print(name[3])
```

Output: 'a'

Note: Indexing ~~starts~~ always starts from 0.
 So, Savel
 0 1 2 3 4 → Index.

- * The Immutable nature of String in python.
 → Once a string is created in the memory, its character cannot be modified, added or deleted.

Example:

```
name = "Savel"
```

```
name[1] = "k"
```

```
print(name)
```

Output: TypeError.

⑧ String Concatenation:

→ it is the process of joining two or more strings together into a single new string.

Example:

```
a = "Savel"
```

```
b = "Kaini"
```

```
print(a+b)
```

Output = SavelKaini

→ We can also join the variables and text
 a = "Savel"

```
print(a + "Kaini")
```

Output = SavelKaini

⊛ Important Rules for Concatenation:

- + betⁿ two strings joins them directly, with no automatic spacing added.
- + betⁿ strings and number raises `TypeError` unless the number is converted into `str`.
- F-string automatically converts values placed inside `{ }` into text.
- An f-string must start with the letter 'f' immediately before the opening quote.

8) Escape Characters

→ An escape sequence begins with backslash "\" followed by another character

Example:

<code>Print("Hello\nworld")</code>	<code>Print("Name:\tSaval")</code>
Output: Hello world	Output: Name: Saval

9) Combining Multiple Escape characters:

→ We can combine multiple escape character in python by placing them sequentially within a single string or by combining them using string concatenation

Example:

```
print("Fruits: \n\t- \"Apples\" \n\t- \"Banana\"")
```

Output:

Fruits:

- "Apples"

- "Bananas"

Common Mistakes:

1) Putting a matching quote inside a string without escaping it (eg: "He said 'Hi'") breaks the string

2) Using a single backslash in a file path -

→ The combination like (\U or \t) may be interpreted as an escape sequence -

9) Type Casting

→ It means changing a value from one type to another.

Example:-

age = "20"

age1 = int(age)

So, the text "20" became the real number 20 after using int().

- ⊛ Converting values betⁿ numeric, text and Boolean
- print(int("100")) # text to number
 - print(str(100)) # ~~number~~ number to text
 - print(float(7)) # integer to float
 - print(bool(1)) # number to True/False

Notes:

- int() doesn't rounds a decimal.

Example: int(9.8) # gives 9, not 10

- Comparing a number to its text version

Example: 5 == "5" # this is false

10) Operators:

→ Python operators are special symbols used to perform operations on values and variables.

* Different types of operators

1. Arithmetic operators

→ performs standard mathematical calculations on numeric types like integer and float numbers

Example:

`a = 2`

`b = 3`

`add = a + b` # perform addition (+)

`sub = a - b` # subtraction (-)

`mul = a * b` # multiplication (*)

`div = a / b` # division (/)

`mod = a % b` # modulus generate remainder (%)

`exp = a ** b` # calculate power (**)

2) Logical Operators

→ Primarily used to combine conditional statements and control program flow.

`a = 5, b = 10`

→ `if a = 10 and b = 10 :` # and (both should be true)

→ `elif a = 10 or b = 10 :` # OR (at least one be true)

→ `elif not a = 10 :` # not ($a \neq 10$)

(*) Comparison Operators

$a == b$ # equal to

$a != b$ # not equal to

$a > b$ # greater than

$a < b$ # less than

$a <= b$ # less than equals to

$a >= b$ # greater than equals to

1) Lists:

→ List is a collection of multiple values stored in one variable

Notes:

- List are mutable (changeable) and allow duplicate.
- Lists are written using `[]` brackets.
- Indexing starts from 0.
- Lists can store different datatypes
- Elements can be added, removed and modified.
- List supports indexing, slicing and looping.

* Example:

numbers = `[10, 20, 30, 40]`

Methods:

1. `append()` → Add one element to the end of list.
2. `insert()` → Adds one element at specified Index.
3. `extend()` → Adds multiple elements to the end

- 4) `remove()` → Removes specified Index value
- 5) `pop()` → Removes and return an element specified Index
- 6) `reverse()` → Reverse the order of elements of the list
- 7) `sort()` → Sort list in ascending order by default
- 8) `Index()` → Returns the index of first occurrence of value

13) Random Module:

→ It is a built in python library used to generate random numbers, choices and shuffle.

(*) Import Random to your program

Syntax:

```
import random
```

Example:

```
import random
```

```
number = random.randint(1,10)
```

```
print(number)
```

∴ selects a random number to that range.

- `random.randint(a,b)` - Returns a random integer from a to b (inclusive)
- `random.randrange(a,b)` - Returns a random integer from a range including b .
- `random.choice()` - Select random elements from a sequence.
- `random.sample()` - Selects random multiple unique elements.
- `random.shuffle()` - Randomly rearranges the element of list.

14) Python and function in python

→ Python is a popular high level programming language and has a simple syntax which is close to normal English.

* Python Syntax:

→ It is basically a set of rules that tells us how a program must be written.

Example:

```
# Print hello, world
Print("hello, world")
```

* Diff betⁿ:

• Compiler

1. Translates all code at one time.

2. It is generally faster

3. Reports errors after completion

4) Eg: C, C++

• Interpreter

1. Translates and runs code line by line.

2. It is generally slower.

3. Stops when ^{error} is found.

4) eg: python

⑦ Basics syntax of a function

- `def` → keyword used to define function
- `function_name` → give a name to function
- `()` → parentheses which holds parameters
- `:` → the function body is starting.
- indented code → the lines that run when function called.

Example:-

```
def show-message():
    print("Learning is fun")
```

⑧ Python 'indentation'

→ It means the spaces at the beginning of a line.

Example:

```
marks = 80
```

```
if marks > 40:
```

```
    print("you are pass")
```

↑ # indented line runs only when it's true.

no indented gives error

⑨ Uses of function:

- 1) Avoids writing same code again and again.

Example:

```
Print("saval")
```

```
Print("saval")
```

```
print("saval")
```

VS

```
def name():
```

```
    print("saval")
```

```
name()
```

```
name()
```

```
name()
```

→ written once and can be used multiple times